

EXPERIMENT – 8

Name: K ANUSRI

Roll Number: 2520030432

Section: 8

Title

Variable Expansion and Built-in Command Dispatch

Aim

To detect variable references, expand variable values, handle undefined variables, update tokens, support nested or braced variable references, and test expansion logic. To identify built-in commands, create a dispatch table, execute in-process commands, handle invalid commands, maintain state, and test dispatch logic.

Theory

Variable expansion is the process of identifying references such as \$VAR and \${VAR} in a command or text and replacing them with their corresponding stored values. A variable table can be maintained to create, update, retrieve, and remove variables. If a referenced variable is not defined, the program reports it as an undefined variable.

Built-in commands are commands handled directly by the program instead of creating a separate external process. A dispatch table maps command names to their corresponding functions. This allows commands such as set, unset, echo, pwd, help, and exit to be identified and executed in-process while maintaining the program's variable state.

Algorithm

1. Start the program.
2. Create a variable table to store variable names and values.
3. Create variables and store their values.
4. Detect variable references in the form \$VAR and \${VAR}.
5. Search the variable table for each referenced variable.
6. Replace a defined variable reference with its stored value.
7. Display an appropriate message for an undefined variable.
8. Update an existing variable and test the updated value.
9. Create a dispatch table for built-in commands.
10. Register commands such as set, unset, echo, pwd, help, and exit.
11. Execute built-in commands directly within the program.
12. Maintain the variable state while processing commands.
13. Handle invalid commands with an appropriate message.
14. Test the command dispatch logic.
15. Stop the program.



k_anusri@DESKTOP-TM20BAC



```
k_anusri@DESKTOP-TM20BAQ:~$
k_anusri@DESKTOP-TM20BAQ:~$ #include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>

#define MAX_LINE 512
#define MAX_ARGS 20
#define MAX_VARS 50

typedef struct
{
    char name[50];
    char value[200];
} Variable;

Variable variables[MAX_VARS];
int var_count = 0;

int find_variable(char *name)
{
    int i;

    for (i = 0; i < var_count; i++)
    {
        if (strcmp(variables[i].name, name) == 0)
            return i;
    }

    return -1;
}

void set_variable(char *name, char *value)
{
    int index = find_variable(name);

    if (index >= 0)
    {
        strcpy(variables[index].value, value);
        printf("Variable updated: %s = %s\n", name, value);
    }
    else
    {
        if (var_count < MAX_VARS)
        {
            strcpy(variables[var_count].name, name);
            strcpy(variables[var_count].value, value);
            var_count++;

            printf("Variable created: %s = %s\n", name, value);
        }
    }
}

char *get_variable(char *name)
{
    int index = find_variable(name);

    if (index >= 0)
        return variables[index].value;

    return NULL;
}
```



k_anusri@DESKTOP-TM20BAC



```
void expand_variables(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_LINE - 1)
    {
        if (input[i] == '$')
        {
            char name[50];
            char *value;
            int k = 0;

            i++;

            if (input[i] == '{')
            {
                i++;

                while (input[i] != '}' &&
                    input[i] != '\0' &&
                    k < 49)
                {
                    name[k++] = input[i++];
                }

                name[k] = '\0';

                if (input[i] == '}')
                    i++;
            }
            else
            {
                while ((input[i] >= 'A' && input[i] <= 'Z') ||
                    (input[i] >= 'a' && input[i] <= 'z') ||
                    (input[i] >= '0' && input[i] <= '9') ||
                    input[i] == '_')
                {
                    if (k < 49)
                        name[k++] = input[i];

                    i++;
                }

                name[k] = '\0';
            }

            value = get_variable(name);

            if (value != NULL)
            {
                strcpy(&output[j], value);
                j += strlen(value);
            }
            else
            {
                printf("Undefined variable: %s\n", name);
            }
        }
        else
        {
            output[j++] = input[i++];
        }
    }
}
```



k_anusri@DESKTOP-TM20BAC



```
    }
    output[j] = '\\0';
}

void builtin_pwd()
{
    char cwd[300];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("Current Directory: %s\\n", cwd);
}

void builtin_echo(char *args[], int count)
{
    int i;

    for (i = 1; i < count; i++)
        printf("%s ", args[i]);

    printf("\\n");
}

void builtin_set(char *args[], int count)
{
    int i;

    if (count == 1)
    {
        for (i = 0; i < var_count; i++)
            printf("%s = %s\\n",
                    variables[i].name,
                    variables[i].value);

        return;
    }

    if (count >= 3)
        set_variable(args[1], args[2]);
    else
        printf("Usage: set NAME VALUE\\n");
}

void builtin_unset(char *args[], int count)
{
    int index;
    int i;

    if (count < 2)
    {
        printf("Usage: unset NAME\\n");
        return;
    }

    index = find_variable(args[1]);

    if (index >= 0)
    {
        for (i = index; i < var_count - 1; i++)
            variables[i] = variables[i + 1];

        var_count--;

        printf("Variable removed: %s\\n", args[1]);
    }
}
```

```
}
else
{
    printf("Variable not found: %s\n", args[1]);
}
}

void builtin_help()
{
    printf("\nBuilt-in Commands\n");
    printf("-----\n");
    printf("set NAME VALUE - Create or update variable\n");
    printf("unset NAME - Remove variable\n");
    printf("echo TEXT - Display text / expanded values\n");
    printf("pwd - Display current directory\n");
    printf("help - Display built-in commands\n");
    printf("exit - Exit the program\n");
}

typedef struct
{
    char *name;
    void (*function)(char *[], int);
} Builtin;

void run_pwd(char *args[], int count)
{
    builtin_pwd();
}

void run_help(char *args[], int count)
{
    builtin_help();
}

void run_exit(char *args[], int count)
{
    printf("Exiting shell...\n");
    exit(0);
}

Builtin dispatch_table[] =
{
    {"pwd", run_pwd},
    {"help", run_help},
    {"exit", run_exit}
};

int dispatch_count = 3;

int execute_builtin(char *args[], int count)
{
    int i;

    if (strcmp(args[0], "set") == 0)
    {
        builtin_set(args, count);
        return 1;
    }

    if (strcmp(args[0], "unset") == 0)
    {
        builtin_unset(args, count);
        return 1;
    }
}
```

```
}

if (strcmp(args[0], "echo") == 0)
{
    builtin_echo(args, count);
    return 1;
}

for (i = 0; i < dispatch_count; i++)
{
    if (strcmp(args[0], dispatch_table[i].name) == 0)
    {
        dispatch_table[i].function(args, count);
        return 1;
    }
}

return 0;
}

int main()
{
    char *path;
    char line[MAX_LINE];
    char expanded[MAX_LINE];
    char *args[MAX_ARGS];
    char *token;
    int count;

    printf("=====\n");
    printf(" VARIABLE EXPANSION AND BUILT-IN DISPATCH\n");
    printf("=====\n");

    printf("\n1. VARIABLE REFERENCE TEST\n");
    printf("-----\n");

    set_variable("NAME", "Mohith");
    set_variable("CITY", "Hyderabad");

    printf("\nOriginal: Hello $NAME from $CITY\n");

    expand_variables(
        "Hello $NAME from $CITY",
        expanded
    );

    printf("Expanded: %s\n", expanded);

    printf("\n2. BRACED VARIABLE TEST\n");
    printf("-----\n");

    expand_variables(
        "Welcome ${NAME}!",
        expanded
    );

    printf("Expanded: %s\n", expanded);

    printf("\n3. UNDEFINED VARIABLE TEST\n");
    printf("-----\n");

    expand_variables(
        "Value of $UNKNOWN",
        expanded
    );
}
```



k_anusri@DESKTOP-TM20BAC



```
);

printf("Result: '%s'\n", expanded);

printf("\n4. VARIABLE UPDATE TEST\n");
printf("-----\n");

set_variable("NAME", "Student");

expand_variables(
    "Updated value: $NAME",
    expanded
);

printf("Expanded: %s\n", expanded);

printf("\n5. BUILT-IN DISPATCH TABLE\n");
printf("-----\n");

printf("Registered built-ins:\n");
printf("set\n");
printf("unset\n");
printf("echo\n");
printf("pwd\n");
printf("help\n");
printf("exit\n");

printf("\n6. IN-PROCESS COMMAND TEST\n");
printf("-----\n");

{
    char *test1[] = {"pwd"};
    execute_builtin(test1, 1);
}

{
    char *test2[] = {"echo", "Hello", "Student"};
    execute_builtin(test2, 3);
}

printf("\n7. INTERACTIVE COMMAND TEST\n");
printf("-----\n");
printf("Enter commands below.\n");
printf("Examples: set AGE 20, echo $AGE, pwd, help\n");
printf("Type exit to stop.\n\n");

while (1)
{
    printf("mini-shell> ");

    if (fgets(line, sizeof(line), stdin) == NULL)
        break;

    line[strcspn(line, "\n")] = '\0';

    if (strlen(line) == 0)
        continue;

    expand_variables(line, expanded);

    count = 0;
    token = strtok(expanded, " ");

    while (token != NULL && count < MAX_ARGS - 1)
```

Execution Screenshots

1. Variable reference, braced variable, undefined variable, and variable update tests

```
=====
VARIABLE EXPANSION AND BUILT-IN DISPATCH
=====

1. VARIABLE REFERENCE TEST
-----
Variable created: NAME = Mohith
Variable created: CITY = Hyderabad

Original: Hello $NAME from $CITY
Expanded: Hello Mohith from Hyderabad

2. BRACED VARIABLE TEST
-----
Expanded: Welcome Mohith!

3. UNDEFINED VARIABLE TEST
-----
Undefined variable: UNKNOWN
Result: 'Value of '

4. VARIABLE UPDATE TEST
-----
Variable updated: NAME = Student
Expanded: Updated value: Student
```

2. Built-in dispatch table and in-process command execution

5. BUILT-IN DISPATCH TABLE

Registered built-ins:

set
unset
echo
pwd
help
exit

6. IN-PROCESS COMMAND TEST

Current Directory: /home/mohith_07
Hello Student

7. INTERACTIVE COMMAND TEST

Enter commands below.

Examples: set AGE 20, echo \$AGE, pwd, help
Type exit to stop.

mini-shell>

Result

The program was executed successfully. Variable references were detected and expanded, braced variables were handled, undefined variables were identified, and existing variables were updated. A built-in command dispatch table was created and built-in commands were executed in-process while maintaining program state.